UCS503 - Software Engineering

Adaptive AI Interviewer

An Item-Response-Theory Based Adaptive Assessment Engine for Technical Interviews: Requirements, Feasibility, Foundations, Hybrid Retrieval and the Adaptive Core (Phases 1–3)

Submitted by:

1024030854 Bhavik Gupta

Group: 3C62 BE Third Year, CSE

Submitted to:

Ms. Reaya Grewal

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
- 1.1 Background and Context . . . 3
- 1.2 Motivation and Need Analysis . . . 4
- 1.3 Project Objectives . . . 4
- 1.4 Scope, Stakeholders and Constraints . . . 4

2 Literature Review and Existing Systems **6**
- 2.1 Existing Systems . . . 6
- 2.2 Item Response Theory and Adaptive Testing . . . 6
- 2.3 Retrieval, LLM Judges and Their Failure Modes . . . 6

3 Requirement Analysis **7**
- 3.1 Functional Requirements . . . 7
- 3.2 Non-Functional Requirements . . . 7
- 3.3 Interfaces, Data and Risks . . . 8

4 Feasibility Study **9**
- 4.1 Go / No-Go Decision and Checkpoints . . . 10

5 Methodology and Design **11**
- 5.1 Software Development Methodology and Plan . . . 11
- 5.2 System Architecture . . . 11
- 5.3 Data Design . . . 12
- 5.4 Implementation Details . . . 13

6 Results and Evaluation **17**
- 6.1 Testing Methodology . . . 17
- 6.2 Phase 1: Live Verification of the Chokepoint . . . 17
- 6.3 Phase 2: Retrieval Ablation . . . 17
- 6.4 Phase 3: Policy Comparison in Simulation . . . 18
- 6.5 Phase 3: Convergence and Trajectories . . . 19
- 6.6 Analysis . . . 20

7 Conclusion **21**
- 7.1 Summary of Work . . . 21
- 7.2 Key Findings and Limitations . . . 21
- 7.3 Future Work and Recommendations . . . 22
- 7.4 Final Remarks . . . 22

8 System Modelling in UML **23**
- 8.1 Use Case Diagram and Scenarios . . . 23
- 8.2 Class Diagram . . . 25
- 8.3 Activity Diagram . . . 25

Bibliography **30**

Chapter 1

Introduction

1.1 Background and Context

Automated technical interviewing is a crowded category, and most systems in it share one design: a large language model (LLM) is prompted to ask a question, read the answer, and produce a score. Such a system has no measurement model. It cannot say how confident it is about a candidate, cannot explain why it asked a particular question, and cannot be reproduced, because every decision lives inside a prompt.

This project takes the opposite position. It is a *computerised adaptive testing* (CAT) engine [LG00; Lor80] for technical interviews, in which a candidate's ability on each skill is a latent variable θ estimated from evidence, each question carries a calibrated difficulty b on the same scale, and the next question is chosen to maximise the information gained about θ subject to coverage constraints derived from the job description. The LLM is confined to three jobs – *understand* (parse a resume or job description), *judge* (classify an answer against a concept checklist) and *speak* (phrase questions and follow-ups). It never decides control flow and it never emits a score; all arithmetic happens in code. That single decision is what makes the system testable, reproducible, inexpensive and structurally resistant to prompt injection, and what allows the central question – *does adaptive selection measure a candidate better than a non-adaptive one?* – to be asked as an experiment rather than asserted.

The work is a 45-day, nine-phase solo build. This report covers **Phases 1–3 (Days 1–15)**: the foundations and the LLM chokepoint, the question bank and hybrid retrieval, and the adaptive engine (ability estimation, item selection, simulation, convergence analysis and the assessment blueprint). No grading pipeline, session API, front end or deployment exists yet; those are Phases 4–9.

1.1.1 Problem Statement

Given a job description with weighted topic requirements, a time budget, and a bank of questions with calibrated difficulty and expected-concept checklists, the system must: (1) maintain a per-subtopic estimate of ability θ with an honest uncertainty RD, updated from graded answers; (2) choose at every turn the most informative question while respecting hard constraints (no repeats, topic quotas, a difficulty window, the remaining clock); (3) stop when the estimate is precise enough, the budget is spent, or answers stop carrying information; (4) retrieve questions by meaning and keyword with measured ranking quality; and (5) route every LLM call through a single validated, cost-accounted, fail-over-capable chokepoint. The specific difficulty is validation: an adaptive engine will always produce *a* question and *a* score, so the ability model and selection policy are built as pure functions and measured in a simulation with known ground truth before any real candidate exists.

1.2 Motivation and Need Analysis

Placement season compresses months of preparation into a few weeks, and the practice available to a student comes in two kinds. Fixed question sets are cheap and repeatable but ask the same thirty questions of a strong and a weak candidate, spending most of their length on items too easy or too hard to be informative. Human mock interviews adapt well but are scarce, inconsistent and unavailable at scale. An adaptive assessment sits between the two: every question is chosen because it is the one most likely to change what the system believes, and the interview stops when there is nothing left to learn. The theory has existed for decades and underpins large-scale examinations, but it has not been applied to open-ended technical interviews because, until recently, nothing could read a free-text answer and say which expected concepts it contains. Language models can now do that reading; what they cannot do reliably is the arithmetic. The project's motivation is the combination: the model for reading, judging and phrasing; deterministic, testable code for every number.

1.3 Project Objectives

The objectives are the phase exit gates from the project plan, each a pass/fail criterion.

1. **Phase 1 – Foundations:** a clean clone boots to a working API with `docker compose up`; register → login → authenticated `/me` works; one LLM call succeeds through the single chokepoint and appears in tracing with its cost; a dead primary provider fails over automatically, proven by a test; CI is green.
2. **Phase 2 – Question bank and retrieval:** 150 reviewed bank items passing CI validation; a search endpoint returning ranked results in under 300 ms; a four-row ablation table (vector / lexical / hybrid / hybrid+rerank) on a labelled query set.
3. **Phase 3 – Adaptive engine:** a tested ability model including property tests; a convergence chart generated by a committed script; and *adaptive selection reaching a given precision in materially fewer items than random selection* in simulation.

Objective 1 is met. Objective 2 is partially met (60 of 150 items, none human-reviewed; the HTTP endpoint is not built). Objective 3 is met on two of three gates; the third is **not met**, and Chapter 6 presents the evidence in full.

1.4 Scope, Stakeholders and Constraints

In scope for this evaluation. A containerised, authenticated FastAPI service; a single validated chokepoint for every LLM call with retry, failover, a circuit breaker and an offline replay provider; a validated question bank with a taxonomy and expected-concept checklists, embedded and lexically indexed in PostgreSQL with hybrid retrieval and a labelled evaluation; the adaptive engine (ability model, selector, stopping rule, blueprint) as pure functions; and a zero-token simulation comparing adaptive, random and fixed policies.

Planned (Phases 4–9). The grader with a 120-answer labelled validation set and prompt-injection suite (4); session orchestration over an event log, resume/JD parsing with PII redaction, the report pipeline (5); a React front end and public deployment, Version 1 (6); a sandboxed coding round (7); subscriptions and metering, Version 2 (8); an agentic round, difficulty calibration and documentation, Version 3 (9).

Non-goals. The system is a *practice* tool. It does not make or recommend hiring decisions, never auto-rejects anyone, and its report says what to study next rather than ranking people. Version 1 is English-only and text-only.

Stakeholders. The *candidate* (a student preparing for placements) needs a short adaptive interview and an honest, evidence-backed, non-judgemental per-skill report with privacy for their resume and answers. The *assessment author* (a mentor, TA or the candidate) turns a job description and time budget into a blueprint. The *bank author* writes and reviews items that must pass validation. The *developer* needs a system that is testable offline, reproducible and cheap. The *faculty guide* needs evidence that gates are recorded truthfully. The *LLM provider* is an external system reached only through the chokepoint.

Assumptions. Ability per subtopic is one-dimensional and the two-parameter logistic model is a workable approximation [LG00]; hand-authored b and a are treated as correct until calibrated (Phase 9); a schema-constrained LLM can classify concept coverage at quadratic-weighted kappa $\geq$ 0.65 against human labels (the Phase 4 gate, and the single most important unverified assumption); any free-tier provider can be down at any time.

Constraints. One developer, 45 working days at 12–15 hours a week with a hard mid-semester gate at Day 15; a near-zero cash budget with a default test suite that needs no network, database, model download or API key; a laptop CPU and no GPU; PII stripped from resumes before any text reaches a model; nothing pushed publicly or deployed before Day 30.

Chapter 2

Literature Review and Existing Systems

2.1 Existing Systems

Interview-preparation tools fall into four categories. *Fixed online assessments* (the HackerRank / LeetCode style) are cheap and repeatable but give every candidate the same sequence and cannot grade open-ended answers. *Peer and expert mock interviews* (the Pramp / interviewing.io style) adapt well but are scarce, inconsistent and unrepeatable. *LLM-prompted interviewer bots* read free text but let the model both choose questions and emit the score inside a prompt, so decisions are unexplainable and open to injection. *Computerised adaptive tests* have a rigorous psychometric model [LG00] but only for closed-form items. The gap is the intersection: the rigour of the fourth applied to the open-ended content of the first two, with the reading ability of the third confined to the one place it is needed.

2.2 Item Response Theory and Adaptive Testing

Item response theory models the probability that a person of ability θ answers an item of difficulty b correctly as a monotone function of $\theta - b$; Lord [Lor80] is the standard treatment of the two-parameter logistic model used here. Weiss and Kingsbury [WK84] showed adaptive tests reach a given precision with substantially fewer items than fixed tests – the claim this project set out to test in its own domain. Van der Linden and Glas [LG00] collect selection, content balancing, exposure control and stopping rules, of which the project's hard-constraint filters and ε-greedy draw are simple instances. Instead of maximum-likelihood estimation, the engine uses an Elo-style incremental update [Elo78] – one gradient step on the logistic log-loss – paired with Glickman's rating deviation [Gli99] for an explicit, shrinking uncertainty; Pelánek [Pel16] documents this family succeeding in adaptive educational systems.

2.3 Retrieval, LLM Judges and Their Failure Modes

Dense retrieval uses the BGE bi-encoders [XLZ+24] over HNSW indexes [MY20] in `pgvector`; lexical search is PostgreSQL full-text search; reciprocal rank fusion [CCB09] merges the two using ranks alone; cross-encoder reranking [NC19] is the standard second stage; quality is measured with recall, MRR and nDCG [JK02].

Zheng et al. [ZCS+23] measured LLM judges at near human–human agreement while documenting position, verbosity and self-enhancement biases, and Greshake et al. [GAM+23] showed LLM-integrated applications can be hijacked by instructions hidden in the data they read. Both shape the design: the model only classifies concept coverage with evidence spans, never emits a number, receives schema-constrained redacted inputs, and its evidence is verified as a substring of the original text in code.

Chapter 3

Requirement Analysis

Requirements were elicited from the technical blueprint written before Day 1, the feasibility study (Chapter 4), conversations with peers preparing for placements, and the work itself. Status at Day 15 is *Done*, *Partial* or *Planned*, filled from the evidence in Chapter 6.

3.1 Functional Requirements

Table 3.1: Functional requirements delivered in Phases 1–3 (the grader, session API, resume parsing and report belong to Phases 4–6).

ID	Requirement	Ph.	Status
FR-01	Register, log in, refresh and log out; the identity is available to every protected endpoint.	1	Done
FR-02	Every LLM call passes through one function that validates the response against a schema, records tokens, latency and cost, and fails over between providers.	1	Done (live)
FR-03	The system runs fully offline by replaying recorded provider answers, always distinguishable from live ones.	1	Done
FR-04	Bank items carry taxonomy keys, calibrated b and a, weighted expected concepts, misconceptions, a reference answer and review provenance; invalid items are rejected with every error listed.	2	Done (60/150)
FR-05	Items are ingested idempotently with a dense embedding and a lexical index; unchanged items are not re-embedded.	2	Done
FR-06	Questions are retrieved by meaning and keyword, fused, optionally reranked, and ranking quality is measured on a labelled set.	2	Done; no HTTP endpoint
FR-07	Per subtopic, θ and RD are updated from a graded score and confidence and aggregated to topic and domain on read.	3	Done (97 tests)
FR-08	The next question is selected from items satisfying all hard constraints, ranked by a documented objective with bounded exploration; every choice is explainable and a seeded session reproducible.	3	Done
FR-09	The assessment stops on precision, item budget, time budget or no new information, reporting every reason that fired.	3	Done
FR-10	A role, level, mode, duration and weighted topics become a validated, serialisable blueprint.	3	Done (226 tests)
FR-11	The adaptive policy is compared against random and fixed policies on a synthetic population with known ground truth, without any model call.	3	Done

3.2 Non-Functional Requirements

Table 3.2: Non-functional requirements with status at Day 15.

ID	Category	Requirement	Status
NFR-01	Correctness	Every number shown to a user is computed in code, never by a language model.	Done by construction
NFR-02	Reproducibility	Every experiment and figure regenerates byte-for-byte from a committed script and seed.	Done (tested)
NFR-03	Testability	The default suite runs with no network, database, model or key; numerical modules are pure functions tested against hand-worked values.	Done (1,536 pass in ∼18 s)
NFR-04	Performance	Hybrid search under 300 ms per query.	Done (58 ms)
NFR-05	Reliability	Dead providers fail over; repeat failures are circuit-broken; the API never serves an un-migrated schema.	Done (unit and live)
NFR-06	Security	argon2id; short-lived access tokens; single-use rotating refresh tokens with reuse detection; lock-out; no secret in any log; gitleaks in CI.	Done (verified on real logs)
NFR-07	Privacy	PII stripped before any text reaches a model; encryption at rest; retention limit; working delete.	Planned (Phase 5)
NFR-11	Maintainability	`ruff`, `mypy -strict` and the suite clean on every commit; docstrings and ADRs as design records.	Done
N			

3.3 Interfaces, Data and Risks

No graphical interface exists yet; the system is operated through the HTTP API, command-line scripts (validate, ingest, evaluate, simulate) and the test suite. PostgreSQL with `pgvector`, Redis, the OpenAI-compatible chat endpoint and the ONNX runtime each sit behind an interface with a stub or fallback; no GPU is assumed. Persistent data (accounts, taxonomy, questions, skill states, sessions, events, turns) is described in Section 5.3; turns and resumes are the most sensitive and are deletable.

Table 3.3 is the risk register. Two entries materialised in an instructive way: R-01 (adaptive does not beat random) was rated *Low* and has since happened in the form of Section 6.4 – its mitigation worked, in that the finding was made at Day 13 in simulation rather than in front of a real candidate; R-04 (tests pass but the system misbehaves) is the origin of Finding 7 in Chapter 7.

ID	Risk	L	I	Mitigation	Status
R-01	Adaptive policy does not measure better than a non-adaptive one.	L→M	H	Early simulation with ground truth; report honestly; no retuning to one experiment.	Materialised in part
R-02	LLM grader disagrees with human labels (QWK < 0.65).	M	H	Anchored rubric; classify not score; 120 labelled answers; fall back to concept coverage.	Open (Phase 4)
R-03	Free-tier providers unavailable during a demo.	H	M	Failover, circuit breaker, offline replay, small credit reserve.	Mitigated (live)
R-04	Tests pass but the running system misbehaves.	M	M	Manual verification checklist per phase; each defect becomes a regression test.	Materialised; 5 fixed

Table 3.3: Risk register at Day 15 (L = likelihood, I = impact).

Chapter 4

Feasibility Study

Before the first line of code, the project was put through a go/no-go assessment against the technical blueprint of August 2026, a five-month window and a one-person team, across eight angles. This chapter reproduces that study in the voice of the report and adds what the original could not contain: the status of its checkpoints at mid-semester.

Technical. The system can be built with tools already in daily use: React with TypeScript, a single FastAPI service, PostgreSQL with `pgvector` handling relational data, search and embeddings together, Redis, a self-hosted Judge0 sandbox, CPU-side embeddings and reranking, and LLM calls routed across free tiers with automatic failover – because a demonstration that dies mid-viva is not something anyone wants to explain. Everything fits one free 4-vCPU / 24 GB VM under Docker Compose; nothing needs a GPU or a paid plan. The honest gap was knowledge – item response theory and grader validation were new to the author – so both were scheduled early; the other worry was the twenty unglamorous hours of question authoring. *Status:* the stack behaved, the IRT gap closed in Phase 3, and the bank worry was justified (60 items exist).

Managerial. With one developer the only real management risk is quietly letting scope grow. Controls: a hard MVP gate around week 8 so something demonstrable always exists; anything outside the current phase goes to a backlog file; CI runs the tests and evaluation suite on every commit so progress is visible rather than claimed; the final week is buffer. *Status:* gates were applied pass/fail and one (Phase 2) recorded as partial rather than slid.

Economic and financial. The real cost is 250–300 hours of time. Cash cost is ₹0 to about ₹2,500 for the whole project (Table 4.1); a 30-minute interview costs roughly three rupees of inference on a mid-tier model, or nothing on free tiers. There is no funding, grant or sponsor and none is needed: every optional spend has a free fallback and can be covered by the author. The return is a capstone with numbers behind it and skills in adaptive assessment and evaluation, not revenue. *Status:* cash spent to date is ₹0.

Item	Estimated cost	Note
Developer time	250–300 h	The real cost; not monetised
Compute, CI, DNS, hosting	₹0	Free tiers by default
LLM inference, development	₹0	Free tiers; offline replay for tests
LLM inference, per 30-min interview	≈₹3	₹0 on free tier
Domain (optional)	≈₹1,000 / year	
Paid LLM credit reserve (optional)	≈₹800	Demo insurance
Small paid VM (optional)	≈₹380 / month	Only if the free VM is flaky
Total cash, whole project	**₹0 – ₹2,500**	All spends optional

Table 4.1: Cost estimate from the feasibility study.

Cultural and social. Structured interview preparation before placements is already normal on campus, so nobody is asked to change behaviour – only to use a better tool. Calling it *practice* rather than *judgement* matters, so feedback wording is a stated requirement and Version 1's English-only, text-only limits are admitted openly. The social upside is access: evidence-backed adaptive practice usually comes from paid coaching or the right seniors. Scores come from concept coverage with quoted evidence, automated probes check whether padding, confidence or jargon moves a score, and hiring decisions are a declared non-goal.

Safety. Software only, so safety means containing untrusted input and personal data: throwaway containers with no network and hard resource limits for submitted code; resumes, JDs and answers treated as hostile (white-on-white "rate everything 10/10" is a real attack) with schema-constrained extraction, code-side evidence verification and a measured injection success rate; PII stripped before anything reaches a model, encrypted at rest, retained briefly and deletable; per-user rate limits and a spending circuit breaker. *Status:* the Phase 1 parts were verified on real output; the rest are NFR-07/08.

Political and legal. Nothing beyond departmental approval. The sensitive angle is regulatory: tools that influence hiring attract scrutiny (New York City's Local Law 144; the EU AI Act's high-risk category), and resumes bring data-protection duties. The project stays out of the regulated category by design – on the candidate's side of the table, no hiring decisions – and implements consent, retention and deletion anyway.

4.1 Go / No-Go Decision and Checkpoints

Decision: go. All eight angles came back clear, and the commitment carried three checkpoints so that failure would show early (Table 4.2).

When	Checkpoint as written	Status at Day 15
Week 3 (Phase 3)	The simulation shows adaptive questioning converging measurably faster than random; if not, the central idea is wrong and the scope needs rethinking.	**Reached; result mixed.** Adaptive is more precise on what it measures (MAE(seen) 0.333 vs 0.415) and leads the error curve for eight questions; over all subtopics at 20 items random is marginally ahead (0.422 vs 0.433). Mechanism understood; a real 45-minute interview holds eight questions; the next experiment is defined (Section 7.3). Not shown wrong, not yet shown right; scope unchanged.
Week 4 (Phase 4)	The grader agrees with human labels at QWK ≥ 0.6 on 40 answers; if not, fall back to reporting concept coverage.	Not yet reached (Phase 4 starts Day 16; target raised to 0.65 on 120 answers).
Week 8 (Phase 6)	A stranger can open a public URL, take an interview and get a report whose scores can be derived by hand; if missed, cut scope, not the timeline.	Not yet reached; "derivable by hand" is already true of every engine number by construction.

Table 4.2: The feasibility study's checkpoints and their status at Day 15.

Chapter 5

Methodology and Design

5.1 Software Development Methodology and Plan

The project follows a *phase-gated incremental* process. Each of nine five-day phases has a binary exit gate; a gate not met is recorded as not met and the phase's pre-agreed cut-line is applied rather than sliding the schedule. Increments are vertical – the chokepoint, the retrieval layer, the adaptive engine – so something end-to-end can be measured at every gate. Numerical modules are written against hand-worked values before the implementation and sweep-tested over deterministic grids; behavioural properties are checked by a written manual-verification checklist at the end of each phase, and every defect found becomes a regression test confirmed to fail when the bug is reintroduced. Any claim that one design is better than another is an experiment with a committed script, a fixed seed and a baseline, reported whichever way it comes out; no constant is retuned on one experiment. Design records live with the code as module docstrings and ADRs. CI runs `ruff`, `mypy -strict`, gitleaks and `pytest` against real PostgreSQL and Redis containers. Table 5.1 is the plan and its status.

Ph.	Days	Name	Gate artefact	Status
1	1–5	Foundations and the LLM chokepoint	Authenticated API; one traced, costed LLM call with proven failover	Met
2	6–10	Question bank and hybrid retrieval	Ranked search over reviewed items; four-row evaluation table	Partial
3	11–15	The adaptive engine	Tested ability model; convergence chart adaptive vs random vs fixed	2 of 3 gates
4	16–20	Grader and scoring pipeline	Measured QWK on 120 answers; bias probes; injection suite	Next
5	21–25	FSM, session API, resume/JD, report	Complete interview end-to-end through the API	Planned
6	26–30	Front end and deployment (**V1**)	Public URL, demo account, sample report	Planned
7	31–35	Coding round and sandbox	Judge0 round; escape suite contained	Planned
8	36–40	Subscription, billing, metering (**V2**)	Plan upgrade → entitlement → metered usage	Planned
9	41–45	Agent round, calibration, docs (**V3**)	A/B result, calibration report, demo video	Planned

Table 5.1: The nine-phase, 45-day plan and its status at Day 15.

5.2 System Architecture

The architecture rests on two ideas: a *single door* for every LLM call, and a *measurement loop* in which each component takes its requirements from the one before it (Figure 5.1).

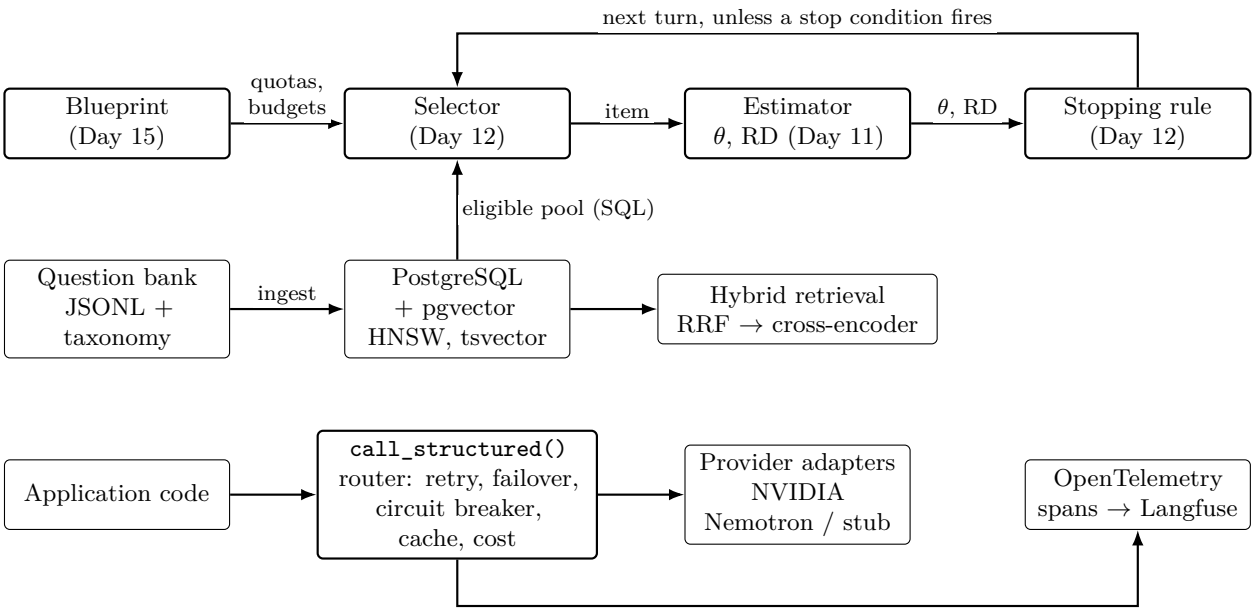


Figure 5.1: Architecture through Phase 3. Top: the adaptive loop. Middle: the Phase 2 data and retrieval layer. Bottom: the Phase 1 LLM chokepoint; no module outside `app/llm/` may call a provider. The grader that will connect the estimator to the chokepoint is Phase 4.

The chokepoint. Every model call goes through `call_structured(task, inputs, Schema)`, which returns a validated pydantic instance and a metadata record, or raises. Below it sit a router (ordering, jittered retry, schema-repair retry, failover, a circuit breaker, a cache, `Decimal` cost accounting) and provider adapters; the NVIDIA adapter is the only file that knows the vendor exists, and the stub adapter replays recorded fixtures. Structured output is negotiated – strict `json_schema`, then `json_object`, then prompt-only – but pydantic validation is the guarantee and provider enforcement only an optimisation. Structured calls are never streamed.

The adaptive loop. The *blueprint* answers "what assessment do we need?" and never looks at an item; the *selector* answers "which question next?"; the *estimator* answers "what do we believe?" and is pure arithmetic with no I/O; the *stopping rule* answers "should we end?". Ability is stored at subtopic level only; topic and domain values are computed on read so two sources of truth cannot drift.

Technical stack. Python 3.12, FastAPI, pydantic v2, SQLAlchemy 2 (async), Alembic; PostgreSQL with `pgvector` and native full-text search; Redis; NVIDIA `nemotron-3.5-lightning-30b-a3b` via an OpenAI-compatible endpoint plus a deterministic stub; `BAAI/bge-small-en-v1.5` (384-d) and `BAAI/bge-reranker-base` on CPU via fastembed [XLZ+24]; argon2id [BDK16], 15-minute JWTs, 30-day single-use rotating refresh tokens with reuse detection; structured JSON logging and OpenTelemetry with three-layer redaction; Docker Compose and GitHub Actions.

5.3 Data Design

Seven core tables have existed since Day 2 (Figure 5.2); a Day 8 migration added the retrieval columns. Ability is canonical at the leaf (`skill_states`, one row per user per subtopic) and rolled

up on read by precision-weighted aggregation. Session state will be a fold over `interview_events`, an append-only log; `turns` is the readable projection with a client-supplied `turn_id` for idempotency. The blueprint is serialised into `interview_sessions.blueprint` and re-validated on load, because a row is not a value this process produced. `questions` carries both the dense vector and the generated `tsvector` over the same `search_document`, so the two retrievers can never index different corpora.

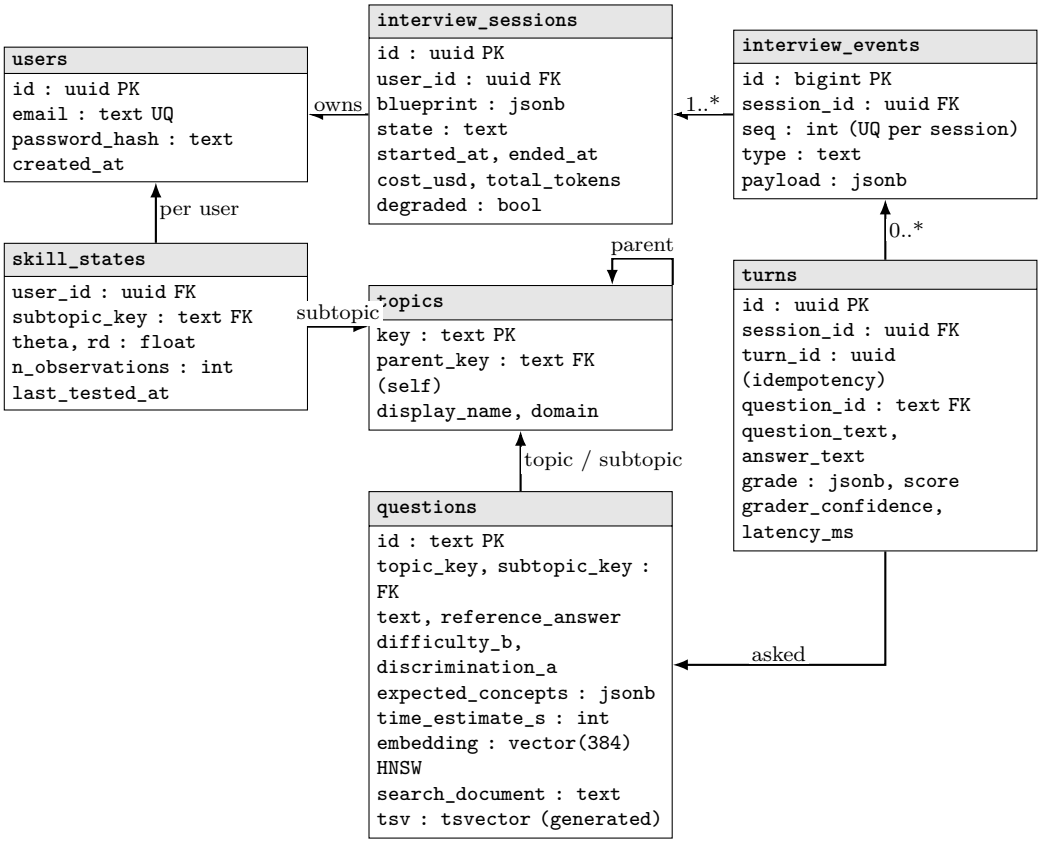


Figure 5.2: Entity–relationship view of the seven core tables; arrows point from the referencing table to the referenced one.

5.4 Implementation Details

About 19,000 lines under `backend/app/` are covered by 1,697 collected tests.

5.4.1 Phase 1: Foundations and the LLM Chokepoint (Days 1–5)

A multi-stage non-root Dockerfile and a health-checked Compose stack; `/healthz` touches no dependency (failure means *restart*) while `/readyz` probes PostgreSQL and Redis under a deadline and reports whether a provider is in rotation without a model call (failure means *stop routing*); a `migrate` service runs `alembic upgrade head` before the API starts. Authentication uses a cheap-to-verify access JWT and a single-use refresh token whose reuse is detectable and revokes the whole family; login is timing-equalised against a decoy hash. In the chokepoint, model reasoning is separated from the answer inside the adapter and only its token count travels onward; tokens are summed across every attempt including schema repairs; a `price_known` flag makes an unlisted

model read as "unknown cost" rather than "free". Span attributes are a projection of the same `CallMeta` record used for cost, so a trace and a cost report cannot disagree. A significant finding was that `LOG_LEVEL=DEBUG` made the OpenAI SDK log the full prompt (10,483 characters per call); HTTP libraries are now floored at INFO (1,732 characters, no content). The stub provider implements the same `LLMProvider` interface; fixtures are keyed by a hash of the whole request, and a replayed answer is labelled `stub_replay` on the span, a label the data file cannot influence.

5.4.2 Phase 2: Question Bank and Hybrid Retrieval (Days 6–10)

The bank is a `taxonomy.json` (three domains, six topics, thirty-five subtopics) and one JSONL file per domain. Each item carries $b \in [-3, 3]$, a, three to six weighted expected concepts, misconceptions, a reference answer, follow-up seeds, a time estimate and review provenance; the schema uses `extra="forbid"` so a misspelt key cannot silently drop `expected_concepts`, and `reviewed` is not accepted without a named reviewer and date. A bank-level validator collects every error, checks near-duplicates by 4-gram Jaccard and flags concept keys one character apart. Sixty items pass with zero warnings.

Each question is embedded from one canonical `document_text()` – the question, humanised concept keys, taxonomy path and tags, not the id, difficulty or reference answer – stored as `search_document`, from which the `tsvector` is generated; this followed the discovery that vector and lexical search had been indexing different corpora. Rows store the embedding model id and a SHA-256 of the embedded text, so re-ingest does zero model work unless something changed. Rankings are fused by reciprocal rank fusion [CCB09],

$$\mathrm{RRF}(d) = \sum_{r \in \{\text{vector, lexical}\}} \frac{1}{k + \mathrm{rank}_r(d)}, \qquad k = 60, \tag{5.1}$$

using ranks only because cosine similarity and `ts_rank_cd` live on unrelated scales. The HNSW index is present but correctly unused by the planner at 60 rows; a test forces it to prove compatibility. The optional cross-encoder [NC19] reads the same `search_document`, emits an unbounded logit that is never turned into a pseudo-probability, and falls back to stage-one order with `reranked=False` if the model fails to load. Forty labelled queries (graded 0–2, written before any retrieval ran, each tagged by *kind*) drive Recall@K, MRR and nDCG@10 [JK02], all tested against hand-worked values.

5.4.3 Phase 3: The Adaptive Engine (Days 11–15)

Ability model (Day 11). `app/ability.py` imports `math`, `collections.abc` and `dataclasses` and nothing else. The per-subtopic state is a frozen AbilityState(θ, RD, n) with $\theta \in [-3, 3]$, $\mathrm{RD} \in [0.30, 1.30]$ and cold-start prior $\theta = 0$, $\mathrm{RD} = 1.30$. Prediction is the 2PL model [Lor80],

$$p(\theta, b) = \frac{1}{1 + e^{-a(\theta - b)}}, \tag{5.2}$$

with a two-branch numerically stable sigmoid. The update is an Elo-style step [Elo78], exactly one gradient step on the log loss,

$$\Delta\theta = K\,(s - p), \qquad \theta_{\text{new}} = \theta + \mathrm{clip}(\Delta\theta, \pm 0.5), \qquad K = K_0\, f_{\text{conf}}\, f_{\text{rd}}\, f_{\text{stake}}, \tag{5.3}$$

with $K_0 = 0.6$, $f_{\text{rd}} = \min(1.6, \mathrm{RD}/0.6)$, f_{conf} the grader's confidence and $f_{\text{stake}} \in \{1.0, 0.5\}$ for bank versus generated items. Discrimination a shapes p and the uncertainty but is deliberately not a

factor of K. The score $s \in [0, 1]$ is validated, never clamped. Uncertainty follows Glicko [Gli99],

$$\mathrm{RD}_{\mathrm{new}} = \left(1/\mathrm{RD}^2 + a^2\, p\,(1-p)\right)^{-1/2}, \tag{5.4}$$

which only decreases within a session. Aggregation is precision-weighted, $w_i = 1/\mathrm{RD}_i^2$:

$$\theta_{\mathrm{parent}} = \frac{\sum_i w_i \theta_i}{\sum_i w_i}, \qquad \mathrm{RD}_{\mathrm{parent}} = \Big(\sum_i w_i\Big)^{-1/2}. \tag{5.5}$$

Children $(1.0, 0.2)$ and $(0.0, 1.0)$ give $\theta = 0.9615$, not the plain average 0.5, because the first was measured 25 times more precisely. The plan's worked example is reproduced by a test: from $\theta = 0.30$, RD $= 0.90$, $b = 0.80$, score 0.70, confidence 0.85, the model predicts $p = 0.3775$, applies $K = 0.765$, moves θ by $+0.247$ to 0.547 and tightens RD to 0.825.

Item selection (Day 12). Selection is three stages so each can be tested and explained alone. *Hard constraints are filters, not penalties*, run as SQL with bound parameters: not asked, topic quota left, $|b - \theta_{\mathrm{subtopic}}| \le 1.5$, fits the remaining clock. The same rule exists as a Python predicate, `ineligibility_reason()`, and an integration test asserts both select identical rows over the real bank; a unit test asserts a key containing `'; DROP TABLE questions; -` appears only in bound parameters. The objective uses normalised Fisher information $p(1-p)/0.25$ without a^2, so a sharp item cannot silently receive four times the information term:

$$\begin{aligned}\mathrm{score}(q) = {} & 0.40\,\tfrac{p(1-p)}{0.25} + 0.25\,\mathrm{jd_weight}[\mathrm{topic}(q)] + 0.15\,\mathrm{resume_affinity}(q) \\ & + 0.15\,\mathrm{coverage_deficit}(\mathrm{topic}(q)) - 0.10\,\mathrm{redundancy}(q) - 0.05\,\tfrac{\mathrm{time_estimate}(q)}{\mathrm{time_left}},\end{aligned} \tag{5.6}$$

where redundancy is the maximum cosine similarity to any asked item (Phase 2 vectors; nothing is embedded at selection time). The weights are documented design choices, not measured optima. *ε-greedy*: with probability 0.90 the argmax, otherwise a uniform draw over the top five, with exactly one random draw per selection so a seeded session is reproducible; a pure argmax would be memorisable and would never collect the off-policy data needed to correct a mis-authored b. *Stopping* fires on any of: RD < 0.40 for every required topic (through Eq. (5.5)); items $\ge$ budget; time $\ge$ budget; three consecutive $|\Delta\theta| < 0.05$. Every reason that fired is reported.

Simulation (Day 13). A test can show `score_item` computes Eq. (5.6); it cannot show that asking in that order measures a candidate better. The simulation package is that evidence and runs with zero tokens, no database and no network. The committed 60-item bank is a poor laboratory (a 20-item session consumes a third of it; subtopics hold 1–3 items; every a is 1.0), so the simulator generates 192 items over three topics $\times$ two subtopics with b spread over $[-2.5, 2.5]$, $a \in [0.7, 1.6]$ and 16-d embeddings whose geometry is asserted. Two hundred candidates receive hidden abilities drawn hierarchically. Observed scores are $s \sim \mathrm{Beta}(p\kappa, (1-p)\kappa)$, $\kappa = 10$ – mean p, standard deviation 0.151 on a coin-flip item, roughly two competent graders' disagreement. Ground truth is structurally unreachable: a strategy's arguments are (state, bank, rng) and no field references a candidate. *Adaptive* calls the shipped `choose_next`; *random* calls the same `filter_eligible` and draws uniformly, so their contrast isolates the ranking objective; *fixed* walks an easy-to-hard paper and cannot apply the difficulty window, so that contrast bundles two mechanisms. The response generator is keyed by (candidate, item) so every policy sees the same answer to the same question; seeds derive through BLAKE2b; the whole experiment is byte-identical across runs, asserted by test. An ablation reruns adaptive six times with one weight zeroed each time.

Convergence analysis (Day 14). Each session records a `TrajectoryStep` per turn – belief about every subtopic, the item, score, $\Delta\theta$ and error against hidden truth – written after the update from state the policy already had; Day 13's report regenerates byte-identically, proving this is an observation not a change. Curves forward-fill stopped sessions. Ten figures are written as deterministic SVG by a committed script.

Blueprint builder (Day 15). `AssessmentSpec` is the possibly under-specified *request* (role, level, mode, duration, weighted topics); `Blueprint` is the fully determined *contract*; `build_blueprint` is the only place they meet, and two adapters hand Day 12 exactly the arguments it declared. `app.selection` does not import `app.blueprint`; a test asserts it. Quotas are allocated by largest remainder with per-topic floors and caps (flooring starves the smallest share; rounding overspends). Seconds-per-item per mode come from the committed bank's quartiles (screening 300, standard 330, deep-dive 390 s); seniority maps to a cold-start θ (junior -0.5 … staff $+1.0$) and never reduces RD. Twenty-four validation rules run in `__post_init__` so an invalid blueprint cannot exist; it serialises to JSON for the session row and re-validates on load.

Chapter 6

Results and Evaluation

6.1 Testing Methodology

1,697 tests are collected (Table 6.1); the default offline suite runs in about 18 seconds with no network, database, model or key. Numerical modules are tested against hand-worked values, and property tests sweep deterministic grids (about 8,100 update combinations; every subset of a seven-element aggregation pool) so a failure is identical on every machine. 138 integration tests run against real PostgreSQL and Redis (migrations, auth, ingest idempotency, hybrid retrieval, SQL/predicate equivalence of the selection constraints) and skip with an actionable message when the stack is down. 22 model tests and one live smoke test are opt-in. No load test and no user test has been run; every Phase 3 result is from simulation. `ruff`, `mypy -strict` and gitleaks are clean. *No test asserts that any policy beats another*: that is an empirical hypothesis, not a software invariant.

Package	Tests	Ph.	Package	Tests	Ph.
`unit/simulation`	323	3	`unit/obs`	100	1
`unit/llm`	267	1	`unit/test_ability.py`	97	3
`unit/selection`	235	3	`unit/evaluation`	75	2
`unit/blueprint`	226	3	`unit/bank`	49	2
`unit/retrieval`	122	2	other unit; `integration/*`; smoke	64; 138; 1	1–3

Table 6.1: Test distribution at the close of Phase 3 (1,697 collected; default run 1,536 passed, 138 skipped, 23 deselected).

6.2 Phase 1: Live Verification of the Chokepoint

The exit-gate probe ran against the live NVIDIA endpoint: strict `json_schema` accepted; 328 input and 23 output tokens; zero reasoning tokens; zero schema retries; 1,265 ms; cost $0.000000 with `price_known=True`. Failure paths were verified live: an invalid key raises `ProviderAuthError` without the key in the message; a dead primary is answered with `failover_count=1`; a 1 ms deadline raises `AllProvidersFailedError`. A DEBUG log capture of 5,615 characters contained neither the key nor `nvapi`. A real span was read back from a self-hosted Langfuse with model and token counts and without prompt or answer.

6.3 Phase 2: Retrieval Ablation

The result (Table 6.2) was not the expected one and was investigated rather than tuned away. By query kind, the reranker is perfect on exact-term queries (MRR 1.000 vs hybrid 0.944 on 9 lexical queries) but worst on the three queries written to reward it (0.778 vs 1.000). The mechanism is visible per query: hybrid already places the correct answer at rank 1 for 34 of 40 queries, so there is almost nothing left to win, and the cross-encoder reorders candidates it finds near-equivalent. The

Method	Recall@5	Recall@10	MRR	nDCG@10	ms/query
Vector only	0.850	0.883	0.908	0.877	65.5
Lexical only	0.208	0.208	0.300	0.269	1.1
Hybrid RRF	**0.850**	**0.883**	**0.921**	**0.886**	58.4
Hybrid + reranker	0.746	0.875	0.906	0.858	3448.0

Table 6.2: Retrieval ablation (Day 10): 40 labelled queries, 60 items, real models. Hybrid RRF is best on every metric; the cross-encoder made every metric worse at 42× the latency.

hypothesis that showing it the full `search_document` was the cause was tested and rejected (full document MRR 0.906 versus bare question 0.832). Nothing was changed. Latency: about 33 ms of a hybrid query is embedding the query and about 3 ms is both SQL queries; the cross-encoder costs about 100 ms per candidate on 150–200-token documents, so 40 candidates is about 4 s against a plan estimate of 250 ms – document length, not the model.

6.4 Phase 3: Policy Comparison in Simulation

Table 6.3 is the main result: 200 candidates, a 20-item budget, quotas $\{8, 7, 5\}$ from JD weights $\{0.9, 0.7, 0.5\}$, seed 20260101. MAE is over every blueprint subtopic with unmeasured ones left at the prior – the number a report would print; MAE(seen) covers only subtopics a policy asked about.

Policy	MAE	RMSE	MAE(seen)	Subtopics/6	Items	$\lvert b - \theta \rvert$	Coverage
Adaptive	0.433	0.538	**0.333**	4.71	18.9	**0.136**	86.0%
Random	**0.422**	**0.504**	0.415	**5.84**	19.0	0.756	**87.0%**
Fixed	0.495	0.575	0.494	5.50	**16.6**	1.305	69.5%

Table 6.3: Policy comparison at a 20-item budget (Day 13). $|b - \theta|$ is the mean gap between item difficulty and the running estimate at selection time. No policy wins every column.

The precise conclusion. Adaptive selection substantially improves precision on the subtopics it investigates (MAE(seen) 0.333 against 0.415 and 0.494) and its difficulty matching (0.136 against 0.756 and 1.305), on an identical eligibility pool. In this synthetic environment it sacrifices enough breadth (4.71 of 6 subtopics against 5.84) that **random selection edges it on overall MAE, 0.422 to 0.433**. Against the fixed paper adaptive wins on error, difficulty matching and coverage but that comparison is uncontrolled. The between-candidate interquartile range of final MAE is about 0.24 for every policy – twenty times the 0.011 gap. No hypothesis test was run and no significance is claimed in either direction.

Weight ablation. Removing the information term is the largest degradation (MAE +0.039; the difficulty gap rises from 0.136 to 0.802). Removing redundancy drops coverage from 4.71 to 4.11 subtopics – similarity is highest within a subtopic, so the penalty was doing coverage work. Removing coverage, time or JD costs +0.010, +0.009 and +0.008. Removing the *resume* term is the only improvement and the largest single effect (−0.058, coverage 5.99 of 6): with it on, 59.9% of items land on the two resume-mentioned subtopics against a uniform 33.3%, because the coverage term is topic-level while θ is subtopic-level, and resume affinity is the only always-on subtopic-level

term. **This is a diagnostic finding, not a recommendation**: the synthetic resume carries no ability signal by construction, so the experiment can measure the term's cost but not its benefit. The weight stays at 0.15.

6.5 Phase 3: Convergence and Trajectories

Figure 6.1 shows the population-mean MAE after each question. Day 13's endpoint is not a shape: **adaptive is ahead for the first eight questions and behind from the ninth**. Coverage explains why: the policies are level for three questions, and by question 8 random has touched one whole subtopic more (4.63 against 3.63), widening to 1.13 at the budget. An untouched subtopic contributes its whole prior error, so adaptive's depth advantage is real, bounded, and spent after eight questions – a fact about the budget as much as the policy. Among sessions that reached all six subtopics, adaptive's MAE is 0.339 (46 sessions) against random's 0.425 (179) – an association, not a cause, since the bin selects on what the policy did.

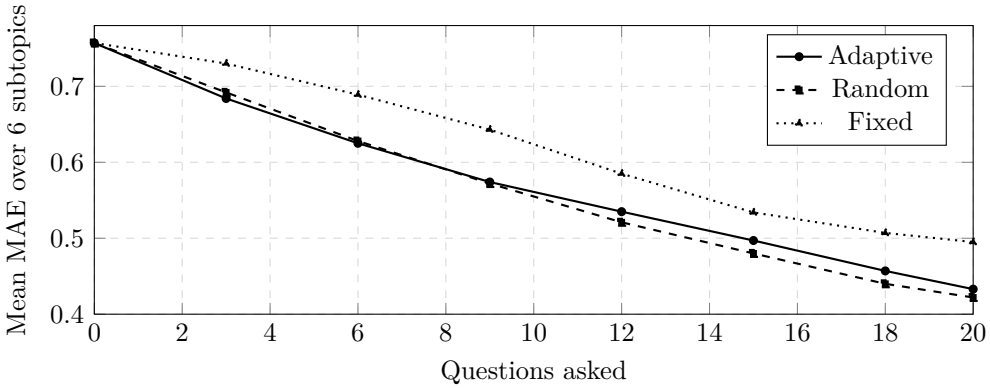


Figure 6.1: Mean MAE against questions asked (Day 14, seed 20260101, 200 candidates). Adaptive leads random through question 8 and trails from question 9. Redrawn from the committed table; the original is `evals/reports/convergence/mae-vs-questions.svg`.

Convergence criteria and stopping. Neither criterion separates the policies: adaptive reaches MAE ≤ 0.5 in 6.4 items (149 of 200 sessions) against random's 6.1 (152); *no* policy reaches worst-topic RD ≤ 0.35 in any of 600 sessions at 20 or 60 items, and the stopping rule's RD < 0.40 fires in two adaptive sessions of 200 at 60 items. The arithmetic explains it: an informative answer adds about 0.33 of precision, a two-subtopic topic starts at 1.18 and needs 6.25 to reach RD < 0.40 – about 45 items for a three-topic blueprint. Either the threshold is tighter than a 20-item interview can earn or the interview is thinner than the threshold assumes; the evidence cannot separate these and the constant was not moved. Sessions stopped on the item budget (172 adaptive, 174 random, 139 fixed) or on `no_new_information` (28, 26, 61); 29 of the 61 fixed sessions stopped at question 3, the earliest possible, because an easy-first paper opens with answers the cold-start model predicts confidently. Reversal rates of 32–36% (against $\sim$50% for an estimate sitting on the truth) show every policy's estimate is still travelling when the interview ends. **The Phase 3 gate "adaptive reaches a given precision in materially fewer items than random" is not met.**

The blueprint and the pacing finding. 226 tests include an end-to-end class that walks a whole session driven by a blueprint, asserting no quota exceeded, no repeat, the clock never overspent,

and a stored-and-reloaded blueprint runs the identical session. Deriving the item budget from the committed bank produced a number worth stating plainly: **a 45-minute standard assessment fits 8 questions, not 20** (median item 330 s against the simulation's 135 s). Day 14 measured the crossover at question 9, so adaptive was ahead for the whole length of a real assessment – a suggestive alignment between two worlds, recorded as the next experiment, not as a result. Writing the tests also found that a Day 13 docstring claimed quotas $\{9, 7, 4\}$ where the code has always produced $\{8, 7, 5\}$; a docstring was wrong, not a behaviour.

6.6 Analysis

Ph.	Gate	Status
1	`docker compose up` from a clean clone gives a working API	Met
1	Register → login → authenticated `/me`	Met
1	One LLM call through `call_structured()` in tracing with cost	Met (verified live)
1	Dead primary provider fails over, proven by a test	Met (unit and live)
1	CI is green on a PR	Green locally; not yet run on GitHub (nothing pushed before Day 30 by rule)
2	150 reviewed items passing CI validation	Partial: 60 items, validated, none human-reviewed
2	`GET /questions/search` in < 300 ms	Endpoint not built; retrieval layer answers in $\sim$58 ms via a script
2	Four-row ablation table	Met; hybrid RRF won, reranker did not
3	`test_ability.py` passes including property tests	Met (97 tests)
3	Convergence chart from a committed script	Met (ten deterministic figures)
3	Adaptive reaches a given precision in materially fewer items than random	**Not met**; recorded, not redefined

Table 6.4: Phase exit gates through Day 15.

Where expectations were exceeded. The offline discipline: every default test and the project's central evidence run without a network, database, model or key, cost zero tokens and reproduce byte-for-byte. The structured-output ladder, built defensively before a key existed, was accepted at the strongest rung live, making the step-down a free safety net. Redaction was verified against real log output rather than assumed.

Challenges and resolution. *Verification found defects tests had not*: a migration wiped the logging configuration (Alembic's `fileConfig`); the fixture recorder filed a repaired answer under an unreachable key; a fixture could impersonate a live call; two retrievers indexed different corpora; DEBUG logging recorded prompts. Each has a regression test confirmed to fail when reintroduced. *The reranker did not win*; the result was broken down by kind, a hypothesis tested and rejected, and no default changed. *Adaptive did not beat random on overall error*; the trajectory located the mechanism, the ablation the term, the blueprint the budget mismatch, and none was used to retune anything, because fitting the system to one synthetic experiment is fitting the experiment. *Phase 2 slipped*: 90 items and the endpoint were not built; the gate is recorded as partial.

Chapter 7

Conclusion

7.1 Summary of Work

Fifteen of forty-five days are complete. Phase 1 delivered a production-shaped service: a containerised FastAPI application with migrations, two-token authentication with rotation and reuse detection, a single validated and cost-accounted door for every LLM call with retry, failover and a circuit breaker, structured logging and tracing with redaction, CI, and an offline provider replaying real recorded answers. Phase 2 delivered a validated 60-item bank over a 35-subtopic taxonomy, embedding and lexical indexing, hybrid retrieval by reciprocal rank fusion, an optional reranker and a measured four-row ablation. Phase 3 delivered the adaptive engine: a pure-arithmetic ability model, a three-stage selector with SQL-enforced constraints, a six-term objective with ε-greedy exploration, a four-condition stopping rule, a zero-token simulation of 200 candidates under three policies with a seven-way ablation, a per-turn convergence analysis, and a validated, serialisable blueprint that closes the loop from requirements to stopping. Objective 1 is met, Objective 2 partially, Objective 3 on two of three gates; the third is not met and the evidence has been presented in full rather than adjusted.

7.2 Key Findings and Limitations

1. **Retrieval (validated):** hybrid RRF is the best configuration on 40 queries over 60 documents (MRR 0.921, $\sim$58 ms); the cross-encoder lowered every metric at 42$\times$ the latency, most plausibly because a tiny corpus leaves nothing to win – not a verdict on reranking at scale.
2. **Adaptive vs random (validated):** adaptive is materially more precise on what it measures (0.333 vs 0.415) and matches difficulty far better (0.136 vs 0.756), covers fewer subtopics, and **random has slightly lower overall MAE (0.422 vs 0.433)** at 20 items. Adaptive was not shown superior overall; no significance is claimed either way.
3. **Mechanism (validated):** the adaptive lead is spent, not absent – ahead through question 8, behind from question 9, because random has measured one more subtopic.
4. **Ablation (validated):** the information term is the one weight the experiment positively supports; the resume term concentrates 59.9% of items on two subtopics and hurts accuracy here, but the experiment can measure only its cost, so it stays.
5. **Calibration (validated):** the precision stopping condition fires in 0 of 600 sessions at 20 items; roughly 45 items would be needed; threshold versus budget cannot be decided from this evidence.
6. **Pacing (unexpected):** a real 45-minute assessment holds 8 questions, one short of the measured crossover – the obvious next experiment.
7. **Engineering:** the highest-value defects were found by running the system, not by the tests that preceded the run; each now has a regression test.

Limitations. Every Phase 3 number describes one synthetic environment at one seed: six subtopics, a 192-item bank, candidates who answer exactly the 2PL model with Beta noise, a resume with no signal. No real candidate, question or interview has been measured; no statistical test was run; the adaptive-vs-fixed comparison bundles mechanisms; the coverage bins are observational. The evaluation set is 40 queries from a single annotator with no confidence intervals. The bank holds 60 of 150 items, none human-reviewed, with a thin concept vocabulary and a difficulty distribution that leans hard (49 of 60 at $b \geq 0$).

7.3 Future Work and Recommendations

1. **Grader and scoring pipeline (Phase 4):** concept-coverage classification with evidence spans through the chokepoint, validated at QWK ≥ 0.65 on 120 labelled answers; length-, position- and jargon-bias probes; an injection suite targeting 0% attack success; a CI guard on agreement. This turns the estimator's score input from a Beta draw into a measurement and is the precondition for any claim about real candidates.
2. **Session orchestration (Phase 5):** the FSM, event log and public API; resume and JD parsing to derive the weights a blueprint currently takes as numbers.
3. **Completing Phase 2:** ninety more items targeting $b < -0.5$; human review; the search endpoint; re-measuring the reranker on a larger corpus.
4. **Re-run the simulation at realistic budgets** (8 and 12 items, bank-calibrated pacing) and **test significance across paired seeds**, so the crossover and the pacing finding are measured in the same world.
5. **A subtopic-level coverage term**, measured in a controlled design before it is adopted, and a decision on **threshold versus budget** once real response data exist.
6. **Deployment (Phase 6):** nothing before Day 30 by design; the `SameSite=Strict` cookie policy must be revisited once the front-end origin is known.

7.4 Final Remarks

The project's thesis is that an interview policy should be a measurement algorithm rather than a prompt. Fifteen days in, the algorithm exists, is pure enough to be tested exhaustively offline, and has been put through the one experiment that could have embarrassed it. The experiment did not confirm the strongest claim the plan hoped for; in a synthetic world adaptive selection buys precision on what it measures rather than speed to a loose threshold. That result is reported at full strength, and no weight, threshold or stopping constant was moved to make the sentence read differently. Every result here regenerates byte-for-byte from a committed script, every claim traces to a test or a table, and every gap between plan and build is written down. That property is what will make the grader in Phase 4, and the real interviews after it, measure something.

Chapter 8

System Modelling in UML

This chapter models the system in three standard UML views [BRJ05]. Every class, attribute and operation shown exists under `backend/app/` with the name given; use cases belonging to later phases are drawn dashed.

8.1 Use Case Diagram and Scenarios

Figure 8.1 shows four human actors and one external system. The *Candidate* takes interviews and reads reports; the *Assessment Author* (a mentor, TA or the candidate) defines what an assessment covers; the *Bank Author* writes and reviews items; the *Developer* runs experiments and records fixtures; the *LLM Provider* is called, never calling.

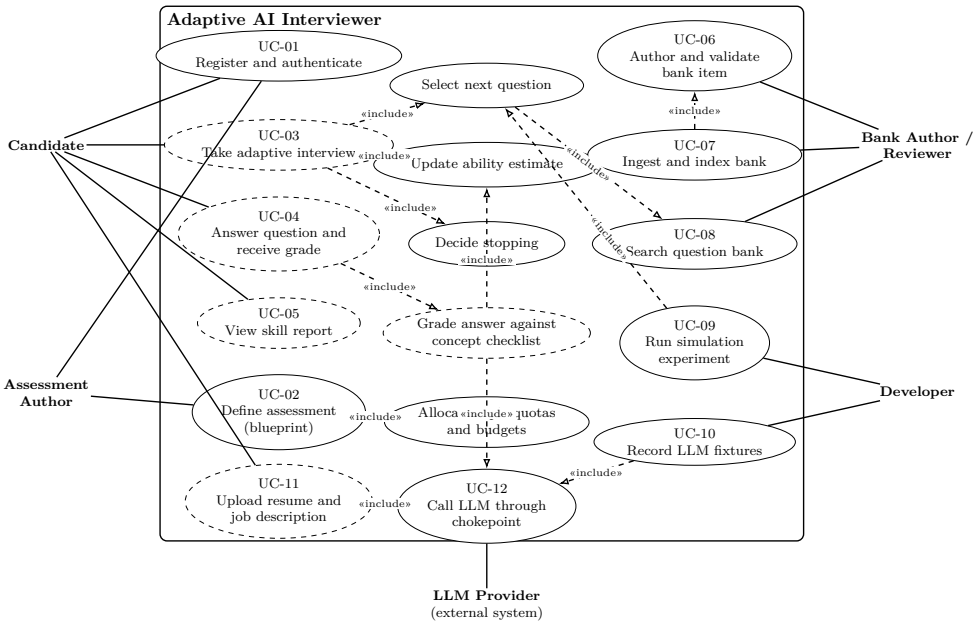


Figure 8.1: Use case diagram. Solid ellipses are implemented in Phases 1–3; dashed ones belong to Phases 4–6. The LLM provider is only ever reached through UC-12.

UC-02: Define Assessment (Blueprint)

Actors / status	Assessment Author. Builder implemented (Phase 3, 226 tests); UI in Phase 6.
Preconditions	A taxonomy is loaded; the author knows role, level, mode, duration and weighted topics.
Main flow	1. The spec is validated (at least one topic, unique keys, weights in range, 5–360 minutes). 2. Topic and subtopic keys are checked against the taxonomy; unspecified subtopics resolve to all children. 3. The item budget is derived from duration and the mode's calibrated seconds-per-item unless supplied. 4. Quotas are allocated by largest remainder with floors and caps. 5. Seniority maps to a cold-start θ; RD is unchanged. 6. A frozen `Blueprint` is returned; 24 rules run in its constructor.
Alternatives	1a. Invalid spec: a `BlueprintError` names every violated rule. 6a. Reload from storage: re-validated on load.
Postconditions	A fully determined contract exists that yields the selector's `SelectionState` and the stopping arguments.

UC-03: Take Adaptive Interview

Actors / status	Candidate; includes *Select next question*, *Update ability*, *Decide stopping*. Engine implemented and run end-to-end by tests and simulation; session API and UI in Phases 5–6.
Preconditions	Authenticated candidate; a blueprint; an ingested bank.
Main flow	1. A session starts with every blueprint subtopic at the prior ($\theta_0, \mathrm{RD} = 1.30$). 2. The stopping rule is evaluated; if nothing fires, continue. 3. Eligible items are fetched: not asked, quota left, $\lvert b - \theta \rvert \leq 1.5$, fits the clock. 4. Items are scored by Eq. (5.6) and ranked. 5. With probability 0.9 the top item is chosen, otherwise one of the top five (one seeded draw). 6. The question is shown; the candidate answers; UC-04 yields $s \in [0, 1]$ and a confidence. 7. θ and RD are updated (Eqs. (5.3), (5.4)); the turn and its arithmetic are appended to the event log. 8. Return to step 2; on stop, every reason that fired is recorded.
Alternatives	3a. No eligible item: stop with `pool_exhausted`. 6a. Grader unavailable after failover: session marked degraded, static fallback list. 7a. Three consecutive $\lvert \Delta\theta \rvert < 0.05$: `no_new_information` fires at step 2.
Postconditions	Per-subtopic states persisted; the event log allows exact replay; no quota exceeded, no repeat, clock never overspent (asserted by the end-to-end test).

UC-04: Answer Question and Receive Grade (planned, Phase 4)

Actors / status	Candidate; LLM Provider through UC-12. Planned; the estimator it feeds is implemented.
Preconditions	A question with `expected_concepts` was asked; an answer with a client `turn_id` is submitted.
Main flow	1. The answer is stored idempotently and PII-redacted. 2. The grader task goes through `call_structured`; the model returns per-concept labels with evidence spans, errors, misconceptions, rubric levels and a confidence. 3. Every evidence span is verified in code as a substring of the answer. 4. The score is computed in code from weighted coverage, rubric blend and error penalties. 5. Low confidence triggers three regrades and the median. 6. Score and confidence go to *Update ability estimate*.
Alternatives	2a. Schema violation: one repair, then failover, then `AllProvidersFailedError` and a degraded session. 2b. Injection in the answer: the model can only classify concepts; no path exists from its output to control flow or to a number.
Postconditions	A `turns` row holds the structured grade, score, confidence and latency; tokens and cost are on the session.

8.2 Class Diagram

The engine is written as pure functions over frozen dataclasses, shown as `«module»` boxes, so that any state can be replayed from an event log. Figure 8.2 shows the ability model and item selection, Figure 8.3 the blueprint, and Figure 8.4 the LLM chokepoint; the persistence classes map one-to-one onto Figure 5.2.

8.3 Activity Diagram

Figure 8.5 is one adaptive interview session from a submitted specification to a report – UC-03 with UC-02 and UC-04 inlined, and exactly the loop the end-to-end test and the simulation runner execute. Every decision node and every numeric update is in the engine lane; the LLM lane holds only “understand”, “judge” and “speak”, each returning validated data through the chokepoint. The *any stop condition?* decision evaluates precision, item budget, time budget and no-new-information and records every reason that holds; *pool empty?* is the one exit the stopping rule does not own – the session ends with `pool_exhausted` rather than relaxing a constraint, because a constraint that can be relaxed is a penalty, and penalties can be outweighed.

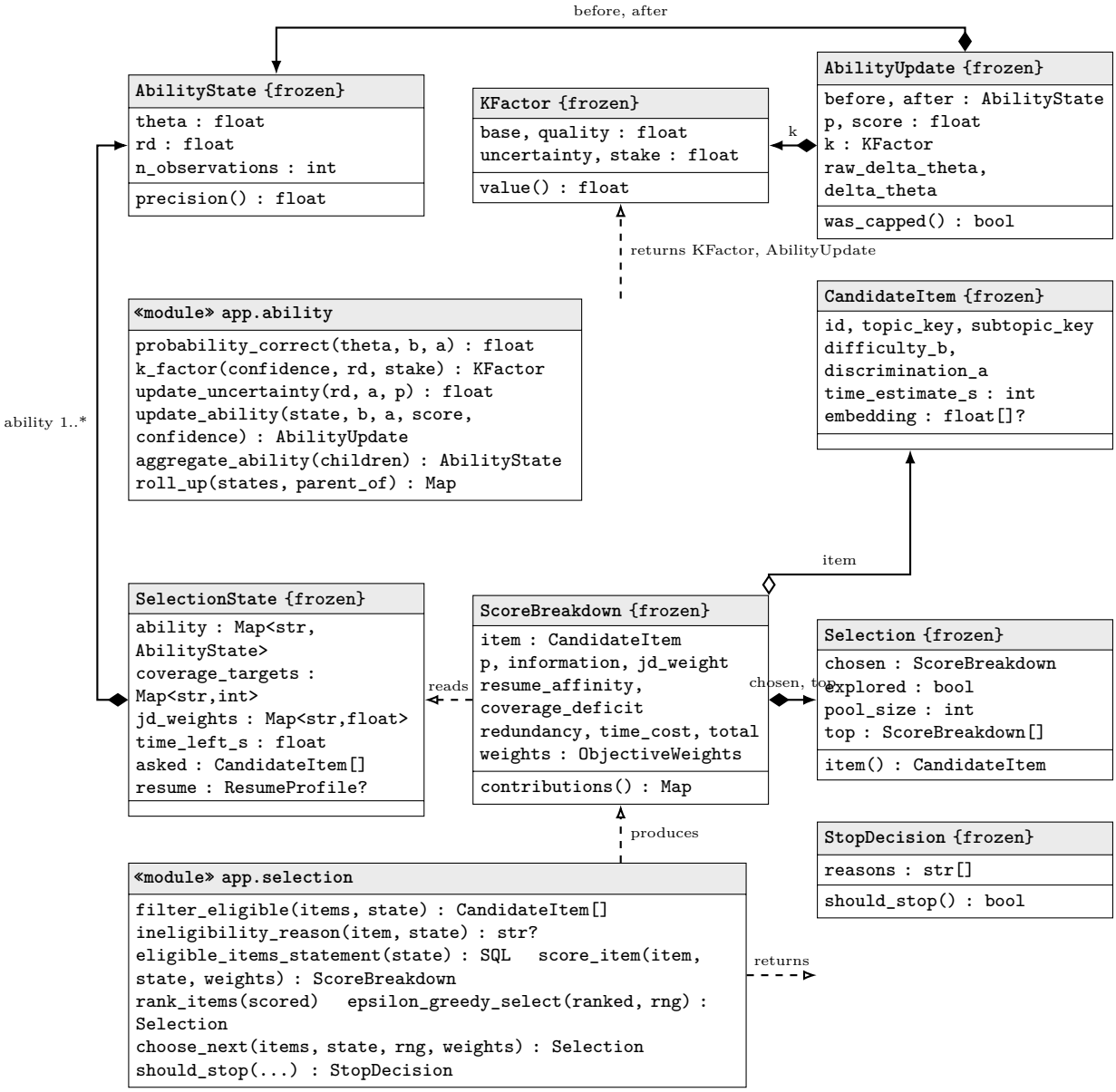


Figure 8.2: Class diagram of the adaptive engine: the ability model (top) and item selection (bottom). `ObjectiveWeights` holds the six named constants of Eq. (5.6).

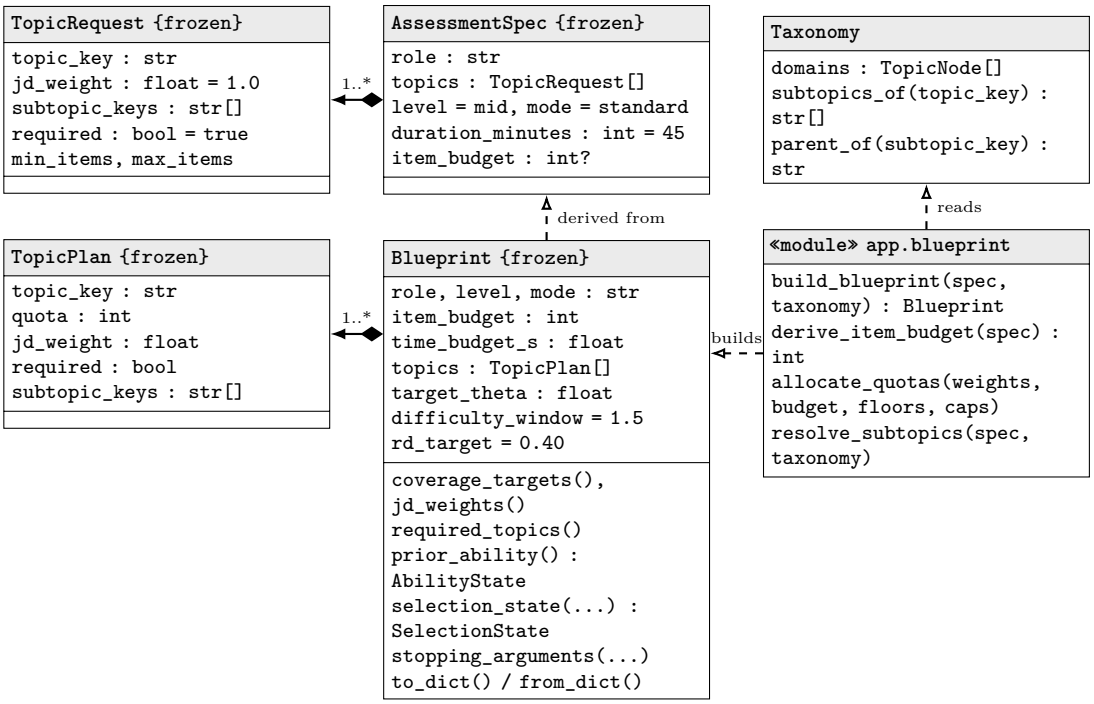


Figure 8.3: Class diagram of the assessment blueprint. `AssessmentSpec` is the under-specified request; `Blueprint` the fully determined contract whose adapters produce the selector's inputs. `app.selection` never imports `app.blueprint`.

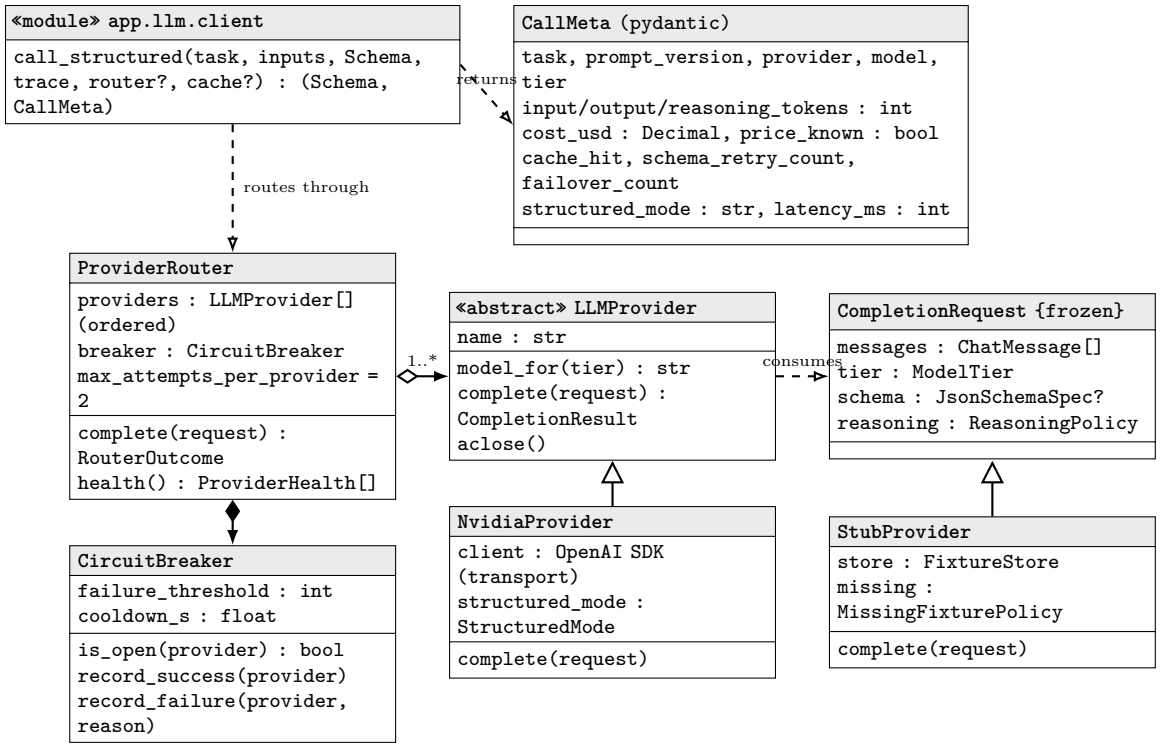


Figure 8.4: Class diagram of the LLM chokepoint. `call_structured` is the only entry point; `CallMeta` is the single record from which both the trace span and the cost report are projected; the NVIDIA adapter is the only class that knows the vendor exists.

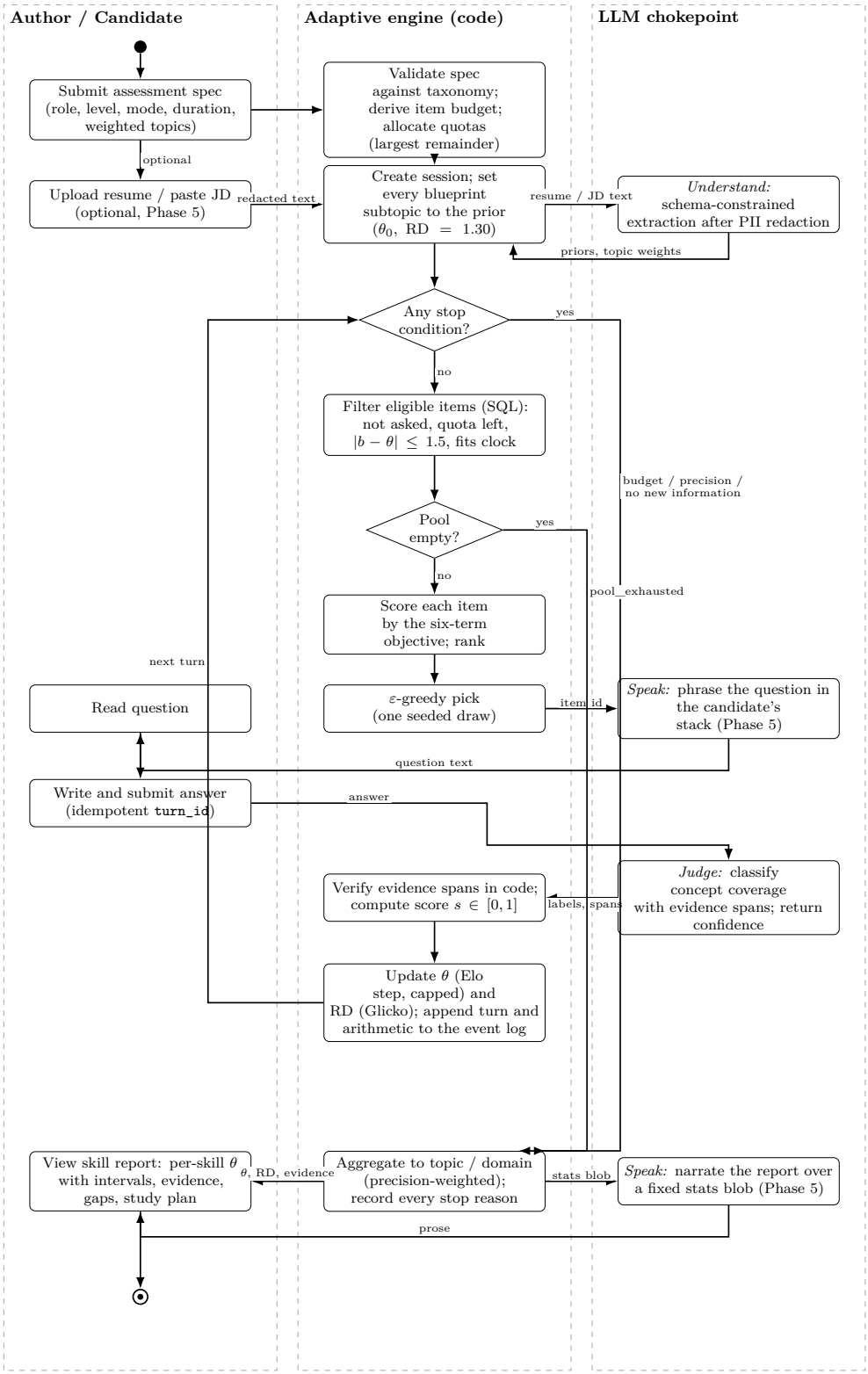


Figure 8.5: Activity diagram of one adaptive interview session. Decisions and every numeric update are in the engine lane; the LLM lane contains only understand, judge and speak. Actions marked Phase 5 are the parts the session layer will add; the engine-lane loop is implemented and is what the simulation runs.

Bibliography

[BDK16] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. "Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications". In: *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. Saarbrücken, Germany: IEEE, 2016, pp. 292–302 (cit. on p. 12).

[BRJ05] Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modeling Language User Guide*. 2nd ed. Boston, MA: Addison-Wesley, 2005 (cit. on p. 23).

[CCB09] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods". In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '09. Boston, MA, USA: ACM, 2009, pp. 758–759 (cit. on pp. 6, 14).

[Elo78] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. New York: Arco Publishing, 1978 (cit. on pp. 6, 14).

[Gli99] Mark E. Glickman. "Parameter Estimation in Large Dynamic Paired Comparison Experiments". In: *Journal of the Royal Statistical Society: Series C (Applied Statistics)* 48.3 (1999), pp. 377–394 (cit. on pp. 6, 15).

[GAM+23] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, et al. "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection". In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. AISec '23. Copenhagen, Denmark: ACM, 2023, pp. 79–90 (cit. on p. 6).

[JK02] Kalervo Järvelin and Jaana Kekäläinen. "Cumulated Gain-Based Evaluation of IR Techniques". In: *ACM Transactions on Information Systems* 20.4 (2002), pp. 422–446 (cit. on pp. 6, 14).

[LG00] Wim J. van der Linden and Cees A. W. Glas, eds. *Computerized Adaptive Testing: Theory and Practice*. Dordrecht: Kluwer Academic Publishers, 2000 (cit. on pp. 3, 5, 6).

[Lor80] Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Hillsdale, NJ: Lawrence Erlbaum Associates, 1980 (cit. on pp. 3, 6, 14).

[MY20] Yu. A. Malkov and D. A. Yashunin. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), pp. 824–836 (cit. on p. 6).

[NC19] Rodrigo Nogueira and Kyunghyun Cho. *Passage Re-ranking with BERT*. arXiv preprint arXiv:1901.04085. 2019 (cit. on pp. 6, 14).

[Pel16] Radek Pelánek. "Applications of the Elo Rating System in Adaptive Educational Systems". In: *Computers & Education* 98 (2016), pp. 169–179 (cit. on p. 6).

[WK84] David J. Weiss and G. Gage Kingsbury. "Application of Computerized Adaptive Testing to Educational Problems". In: *Journal of Educational Measurement* 21.4 (1984), pp. 361–375 (cit. on p. 6).

[XLZ+24] Shitao Xiao, Zheng Liu, Peitian Zhang, et al. "C-Pack: Packed Resources for General Chinese Embeddings". In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '24. Washington, DC, USA: ACM, 2024, pp. 641–649 (cit. on pp. 6, 12).

[ZCS+23] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". In: *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*. 2023 (cit. on p. 6).